

Webstack Agent Harness

Arnés de Ingeniería de Software para Desarrollo Guiado por Agentes Autónomos

Stack Tecnológico: Node.js 22 LTS, TypeScript 5, Fastify, TypeBox, Drizzle ORM, PostgreSQL 16 (tmpfs), Vitest, Stryker Mutator, fast-check, Schemathesis, Gitleaks.

Entorno Operativo: WSL2 / Ubuntu Linux con Docker Compose.

Filosofía de Arquitectura: Aislamiento estricto de capas, ejecución atómica, verificación determinista y bloqueo preventivo ante la ausencia de dependencias.

1. Filosofía Operativa y Diseño Fail-Closed

El arnés responde a un problema fundamental en la generación de software asistida por inteligencia artificial: los agentes tienden al *sobreajuste* (tampering), relajan aserciones cuando encuentran obstáculos y cometen errores silentes de integración si el sistema no bloquea la ejecución de forma binaria.

Principio de Falla Cerrada (Fail-Closed): Si una herramienta de seguridad o comprobación no está instalada, no está configurada o arroja un código de salida distinto de cero, el pipeline aborta de inmediato. Nunca se asume éxito por omisión.

Para asegurar la velocidad de iteración sin degradar la confiabilidad, la verificación se estructura en **cuatro niveles concéntricos** coordinados mediante `justfile`:

Nivel	Comando	Objetivo Temporal	Propósito y Alcance
Nivel 1	<code>just gauntlet-fast</code>	< 5 segundos	Hook de pre-commit. Validación de tipos (<code>tsc</code>), linter (<code>eslint</code>), pruebas unitarias y análisis de secretos en staging (Gitleaks).
Nivel 2	<code>just gauntlet</code>	< 30 segundos	Verificación estructural. Arquitectura de módulos (<code>depcruise</code>), Postgres en RAM, detección de schema drift, reversibilidad de migraciones (Up/Down), matriz de auth y tests de integración con detector N+1.
Nivel 3	<code>just gauntlet-full</code>	< 2-3 minutos	Puerta de enlace pre-merge. Property testing (1,000 iteraciones), fuzzing de contratos OpenAPI (Schemathesis), semillas deterministas (SHA-256), suite hold-out sellada y análisis de mutación del diff (Stryker).
Nivel 4	<code>just audit</code>	Auditoría nocturna	Análisis de mutación exhaustivo sobre todo el código fuente del proyecto.

2. Catálogo y Especificación de los 8 Gates

Gate 1: Migraciones Reversibles Up/Down

Garantiza que toda modificación DDL en la base de datos pueda revertirse limpiamente sin dejar residuos estructurales. El script `scripts/test-migrations-reversible.sh` aplica en una base efímera todas las migraciones `.sql`, luego aplica los rollbacks `.down.sql` en orden inverso, y compara mediante `pg_dump` que el esquema resultante sea idéntico al original antes de volver a aplicar las migraciones.

Gate 2: Detección de Schema Drift

Evita que existan modelos en TypeScript (`src/infra/db/schema.ts`) sin migración SQL versionada. El script `scripts/test-db-drift.sh` toma una instantánea del estado de Git, corre `drizzle-kit generate` y compara si el árbol de trabajo cambió. Si se generan archivos nuevos o se altera el diario de migraciones, aborta con código 1.

Gate 3: Conformidad de Contratos OpenAPI (Fuzzing con Schemathesis)

Valida que la especificación OpenAPI generada por Fastify coincida estrictamente con el comportamiento real del servidor. Schemathesis corre en contenedor Docker enviando casos límite aleatorios. Verifica que no existan errores 500, que todos los códigos devueltos (ej. 200, 401) estén declarados y que los payloads cumplan con los esquemas TypeBox.

Gate 4: Matriz de Autenticación Fail-Closed

Prueba de arquitectura ejecutada en Vitest (`tests/architecture/auth-matrix.test.ts`). Intercepta el hook `onRoute` de Fastify y analiza todas las rutas registradas. Si una ruta no cuenta con el middleware `requireAuth` y tampoco está marcada explícitamente como `config.isPublic = true`, el guantelete falla.

Gate 5: Integridad de Hold-Out Tests

Previene la alteración maliciosa o descuidada de pruebas por parte de un agente (*test tampering*). La suite `tests/holdout/` contiene aserciones de invariantes de seguridad. El archivo `.holdout.sha256` contiene la firma criptográfica de los tests. Si el script `scripts/test-holdouts.sh` detecta un cambio de un solo bit en la suite, bloquea el avance.

Gate 6: Detección de Consultas N+1

Instrumenta el cliente `postgres.js` mediante un listener de depuración síncrono. La función `assertMaxQueries(max, callback)` audita el número exacto de sentencias SQL disparadas por una operación. Si un agente resuelve un listado iterando en bucle en vez de usar consultas por lotes (`batch/inArray`), el test falla mostrando la traza de las consultas.

Gate 7: Determinismo de Seeds y Fixtures

Garantiza que la base de datos de desarrollo y testing sea reproducible bit a bit. El script `scripts/test-seed-determinism.sh` siembra dos bases de datos efímeras independientes y calcula el SHA-256 de las sentencias `INSERT` extraídas mediante `pg_dump`. Prohíbe identificadores aleatorios o fechas dinámicas (`Date.now()`).

Gate 8: Prevención de Fuga de Credenciales

Integrado directamente en `gauntlet-fast`. Evalúa cada commit contra Gitleaks (`gitleaks protect -staged`). Si se detectan tokens de AWS, claves privadas o JWTs en el área de preparación, el hook aborta antes de ejecutar el transpilador. En CI, `gitleaks-action` corre de forma independiente sobre el historial completo (`push`) o el rango exacto del PR (`pull_request`), por lo que la cobertura real de este gate no depende del área de staging vacía de un checkout limpio.

3. Las 4 Capas de Enforcement

Los 8 gates definen *qué* se verifica; las capas de enforcement definen *cuándo* y *quién* no puede saltárselo. Cada capa es independiente de las demás: un agente que desactive una no invalida las otras tres.

Capa 1: Hooks del Agente (`.claude/settings.json`)

`PreToolUse` intercepta cada llamada a herramienta del agente antes de ejecutarse (`scripts/hooks/guard-holdouts.sh`) y bloquea cualquier escritura o borrado sobre `tests/holdout/` o `.holdout.sha256` . Si el payload de entrada no es JSON parseable, el hook falla cerrado de inmediato en vez de asumir que la llamada es inofensiva. `Stop` (`scripts/hooks/stop-gate.sh`) impide que el agente declare terminado un turno si `just gauntlet-fast` no sale en verde. `PostToolUse` (`scripts/hooks/lint-on-edit.sh`) da feedback rápido de ESLint tras cada `Edit / Write` sobre un `.ts` , sin bloquear nada.

Solo `exit 2` bloquea `PreToolUse / Stop` en Claude Code. Los dos hooks de seguridad de este harness usaron `exit 1` desde su creación hasta que se detectó y corrigió en una sesión posterior (ver `docs/adr/0001-hooks-de-seguridad-usan-exit-2.md`) — no bloqueaban nada, pese a que el código parecía correcto. Los hooks de **seguridad** (`PreToolUse` , `Stop`) fallan CERRADO; los de **conveniencia** (`PostToolUse`) fallan ABIERTO. Ver `AGENTS.md` para la tabla completa de clasificación.

Capa 2: Git Pre-Commit Hook

Ejecuta `just gauntlet-fast` antes de registrar cualquier commit local. Como `.git/hooks/` nunca se versiona, el hook se define en `scripts/hooks/pre-commit.sh` y se instala en cada clon nuevo mediante `just install-hooks` (incluido en `just setup`), de modo que la Capa 2 no dependa de la máquina donde se generó el repositorio.

Capa 3: CI / GitHub Actions

`.github/workflows/ci.yml` corre en cada push o PR contra `main` : instala dependencias con lockfile congelado, ejecuta `just doctor` para fallar cerrado ante herramientas ausentes, y luego `just gauntlet-full` (Nivel 3 completo) en un contenedor limpio, sin estado previo del agente.

Capa 4: Branch Protection

La rama `main` exige que el check *Level 3 Pre-Merge Gate* de la Capa 3 pase en verde antes de autorizar el merge. Esta capa se configura en GitHub (Settings → Branches) y no vive en el repositorio como código; debe verificarse manualmente que siga activa.

4. Topología de Archivos y Componentes del Sistema

```
.
├── .claude/
│   ├── agents/
│   │   ├── generator.md      # Subagente: único rol con Edit/Write
│   │   └── reviewer.md      # Subagente: revisa diffs con contexto limpio, sin Edit/Write
│   └── settings.json        # Hooks de agente: PreToolUse, Stop, PostToolUse (Capa 1)
├── .dependency-cruiser.js    # Reglas de aislamiento hexagonal de capas
├── .env.example              # Plantilla de variables de entorno
├── .github/workflows/ci.yml  # Pipeline CI: doctor + gauntlet-full (Capa 3)
├── .holdout.sha256           # Sello criptográfico SHA-256 de invariantes (Gate 5)
├── AGENTS.md                 # Protocolo operativo para agentes autónomos
├── docker-compose.yml        # PostgreSQL 16 con data montada en tmpfs (RAM)
├── docs/
│   ├── adr/                  # Decisiones arquitectónicas: contexto + consecuencias
│   └── progress.md           # Handoff entre sesiones (se sobrescribe, no se acumula)
├── drizzle.config.ts         # Configuración de Drizzle Kit
├── justfile                  # Orquestador declarativo: setup, doctor, niveles 1-4
├── package.json              # Scripts npm y dependencias estrictas
├── scripts/
│   ├── doctor.sh             # Diagnóstico fail-closed de herramientas del sistema
│   ├── hooks/
│   │   ├── guard-holdouts.sh # PreToolUse: bloquea escritura sobre hold-outs (Capa 1)
│   │   ├── stop-gate.sh      # Stop: exige gauntlet-fast en verde (Capa 1)
│   │   ├── lint-on-edit.sh   # PostToolUse: feedback rápido de ESLint (falla abierto)
│   │   └── pre-commit.sh     # Fuente versionada del git hook (Capa 2)
│   ├── mutate-diff.sh        # Stryker enfocado exclusivamente en diff git vs main
│   ├── test-contracts.sh     # Levantamiento de Fastify + Fuzzing Schemathesis (Gate 3)
│   ├── test-db-drift.sh      # Comprobación de deriva de esquema (Gate 2)
│   ├── test-holdouts.sh      # Validador de firma e integridad de hold-outs (Gate 5)
│   ├── test-migrations-reversible.sh # Verificación de Up/Down sin residuos (Gate 1)
│   └── test-seed-determinism.sh # Verificación de identidad criptográfica en seeds (Gate 7)
├── src/
│   ├── domain/               # Lógica pura de negocio y funciones invariantes
│   ├── http/
│   │   ├── app.ts            # Fastify con TypeBox, Swagger y hook de autenticación
│   │   └── server.ts         # Entrypoint HTTP del servidor
│   └── infra/
│       └── db/
│           ├── client.ts     # Cliente Drizzle + postgres.js con query listener (Gate 6)
│           ├── migrate.ts    # Ejecutor programático de migraciones Drizzle
│           ├── schema.ts     # Esquema DDL en TypeScript
│           └── seed.ts        # Sembrador de datos determinista (Gate 7)
└── tests/
    ├── architecture/         # Matriz de autenticación (Gate 4)
    ├── helpers/query-counter.ts # Utilidad de aserción de conteo SQL (Gate 6)
    ├── holdout/              # Invariantes sellados anti-tampering (Gate 5)
    ├── integration/          # Tests contra Postgres en RAM (Gate 6)
    ├── property/              # Tests basados en propiedades con fast-check
    └── unit/                  # Pruebas unitarias de ejecución instantánea
```

Recetas Clave del `justfile`

Receta	Propósito
<code>just setup</code>	Bootstrap completo de un clon nuevo: doctor, instalación de dependencias, instalación del git hook, reset de base de datos y Nivel 2 en verde.
<code>just doctor</code>	Verifica fail-closed que todas las herramientas del sistema estén instaladas antes de operar.
<code>just install-hooks</code>	Instala <code>scripts/hooks/pre-commit.sh</code> como <code>.git/hooks/pre-commit</code> en el clon local (Capa 2).
<code>just db-reset</code>	Recrea <code>webstack_dev</code> desde cero y aplica las migraciones oficiales de Drizzle.
<code>just seal-holdouts</code>	Ruta exclusiva humana: recalcula <code>.holdout.sha256</code> tras un cambio intencional a la suite hold-out.
<code>just review-start / review-clean</code>	Despliega o desmonta un git worktree aislado para el rol Reviewer.

5. Guía de Operación para Agentes Autónomos

Al despachar tareas a un agente de código dentro de este repositorio, el agente debe operar bajo el siguiente flujo de trabajo:

1. **Bootstrap:** En un clon nuevo o entorno desconocido, correr `just setup` antes de cualquier otra cosa; no asumir que el entorno ya está inicializado.
2. **Aislamiento de Entorno:** Operar en una rama de trabajo secundaria creada a partir de `main`.
3. **Ciclo Rápido:** Correr `just gauntlet-fast` tras cada modificación de archivo.
4. **Persistencia de Datos:** Al tocar `schema.ts`, ejecutar obligatoriamente `just db-generate`, escribir el archivo `.down.sql` complementario y verificar con `just gauntlet`.
5. **Exposición de Endpoints:** Todo endpoint registrado en `src/http/app.ts` debe declarar su esquema TypeBox, el contrato `401` si requiere credenciales, o `config: { isPublic: true }` si es público.
6. **Criterio de Aceptación:** Una tarea sólo se considera concluida cuando el agente reporte la salida de `just gauntlet-full` con código de salida 0.

Ver `AGENTS.md` en la raíz del repositorio para el protocolo operativo completo, incluyendo los roles Generator y Reviewer.